

Design and Analysis of Algorithms: CI43

Unit-III

Greedy Algorithms

1. Greedy Algorithms

Greedy Algorithms make the best choice at each step, aiming for the overall best solution. They follow a "**take what looks best now**" approach without reconsidering previous choices.

Key Features:

- Builds the solution step by step.
- Always picks the most immediate and obvious benefit.
- Doesn't look ahead to future consequences.
- **Greedy algorithms** aim to find an optimal solution by making the **best choice at each step**—without looking back or considering the overall problem structure deeply.
- They follow the principle of:
👉 "**Take what looks best now.**"

Examples:

- Fractional Knapsack
- Dijkstra's Algorithm

1. Features Greedy Algorithms

-  **Greedy Choice Property**
 - The final solution can be built by **making locally optimal (best immediate) choices**.
 - Once a choice is made, it's **never changed**.
-  **Optimal Substructure**
 - A problem has optimal substructure if an **optimal solution contains optimal solutions to subproblems**.
 - This is crucial for greedy to work correctly.
-  **Builds Solution Step-by-Step**
 - The algorithm **builds up** the final answer one piece at a time.
-  **No Backtracking**
 - Once a decision is made, it's **not revisited**.
 - No “undo” or exploration of alternatives like in dynamic programming or recursion.
-  **Efficient (Fast and Simple)**
 - Greedy algorithms are generally **faster** than brute force or dynamic programming.
 - They use **simple logic and fewer computations**.
-  **Not Always Globally Optimal**
 - Greedy may **fail** to find the best solution for problems that don't fit the greedy strategy.
 - Example: 0/1 Knapsack (greedy fails), while **Fractional Knapsack** works perfectly with greedy.

- **Characteristics of Greedy Algorithms**
- **Easy to Implement** – Simple and straightforward.
- **Fast & Efficient** – Provides quick solutions with good time complexity.
- **Better than Dynamic Programming (in some cases)** – Used when no need to reconsider past choices (e.g., **Dijkstra's Algorithm** over **Bellman-Ford** for non-negative weights).
- **Greedy Choice Property** – Always picks the best local option to reach a global solution.
- **Optimal Substructure** – The best solution contains the best solutions to its subproblems.

Working of Algorithm

Greedy algorithms solve problems by making the **best immediate choice** at each step, hoping to reach the best overall solution.

Steps:

- **Start** – Begin with the initial state.
- **Evaluate** – Check all available options.
- **Choose** – Pick the best option **at the moment** (without looking ahead).
- **Move Forward** – Update the state and repeat.
- **Stop** – Continue until you reach the goal or can't proceed further.

2. Interval Scheduling: The Greedy Algorithm Stays Ahead

Problem Statement

- Select the maximum number of **non-overlapping** activities from **n activities** with given start and end times.
- **Greedy Algorithm (Earliest Finish Time First)**
- **Sort** activities by finish time (ascending order).
- **Select** the first activity and add it to the schedule.
- **Move** to the next activity that starts **after** the last selected one.
- **Repeat** until all activities are checked.

2. Interval Scheduling: The Greedy Algorithm Stays Ahead

The **Greedy Algorithm** ensures an optimal solution by always choosing the activity that **finishes earliest**, allowing room for more selections.

Why It Works?

- **Maximizes available time** for future activities.
- **Prevents overlapping** by always selecting the earliest finishing activity.
- **Ensures optimality** since any other choice would reduce the number of non-overlapping activities.
- This principle helps the **Greedy Algorithm "stay ahead"**, making the best decision at each step to reach an optimal solution.

2. Interval Scheduling: The Greedy Algorithm Stays Ahead

Problem Statement

- Given a set of activities with start and end times, select the maximum number of **non-overlapping** activities.

Greedy Strategy to Solve:

- **Sort** all activities by their **end time** (ascending).
- **Select** the first activity.
- For each next activity, **if its start time $\geq$ end time of last selected activity, select it.**
- Repeat until all activities are considered.

2. Interval Scheduling: The Greedy Algorithm Stays Ahead

Activity	Start	End
A1	1	10
A2	2	3
A3	4	5
A4	6	7

2. Interval Scheduling: The Greedy Algorithm Stays Ahead

If we pick A1 (1–10) first:

A1 overlaps with all others, so we cannot pick anything else.

✓ Total = 1 activity

✓ **If we sort by end time and pick earliest finishing activities:**

Pick A2 (2–3)

Then A3 (4–5)

Then A4 (6–7)

None of these overlap.

✓ Total = 3 activities

Heap Sort

A **heap** is a special **tree-based structure** that looks like a **binary tree** but follows two rules.

Rule 1: Complete Binary Tree

All levels are completely filled except possibly the last.

The last level is filled from left to right.

✓ Rule 2: Heap Order Property

◆ Max-Heap:

Parent node $\geq$ Children

Largest element is at the root (top).

● Min-Heap:

Parent node $\leq$ Children

Smallest element is at the **root**.

Used in **Priority Queues**.

Heap Sort

Types of Heap:

- **Max-Heap:** Parent is always **greater than** its children.
- **Min-Heap:** Parent is always **smaller than** its children.

In Heap Sort, we use a **Max-Heap** to get the **largest element** at the top.

Heap Sort

[90, 85, 70, 60, 85, 65]

- Corresponding Max-Heap Tree:
- This array represents a **Max-Heap**. Let's draw the **binary tree structure** from it.
- We insert elements level by level from **left to right**:

Example:MaxHeap

90 ← index 0

/ \

85 70 ← index 1, 2

/ \ /

60 85 65 ← index 3, 4, 5

Each parent is **greater than** its children → So it's a **Max-Heap**

Heap Sort

Min-Heap Example

Array: [10, 15, 20, 17, 25]

Corresponding Binary Tree:

10

**/ **

15 20

**/ **

17 25

every parent is less than or equal to its children – that's the Min-Heap property!

Heap Sort Algorithm

Heap Sort Algorithm (Max-Heap Based)

◆ Step-by-step:

- **Build a Max-Heap** from the input array.
- **Swap** the root (maximum element) with the last element.
- **Reduce** the heap size by 1.
- **Heapify** the root again to restore the Max-Heap property.
- Repeat steps 2–4 until the heap size is 1.

Note: Formula used for heapify: In a 0-based array representation of a binary heap:

Left child of node i:

$$\text{left}(i) = 2 * i + 1$$

Right child of node i:

$$\text{right}(i) = 2 * i + 2$$

heapify() function

What heapify() does:

Compares a node with its left and right children

If a child is greater (in Max-Heap), swap with the largest

Recursively apply heapify on the affected subtree

Example: Heap Sort using Array and Binary Heap Tree

Let's take a sample array:

Initial array: [4, 10, 3, 5, 1]

Step 1: Build a Max Heap from the array.

Convert the array into a binary tree and rearrange it into a Max Heap (largest element at the root).

heapify() function and Example

What heapify() does:

Compares a node with its left and right children

If a child is greater (in Max-Heap), swap with the largest

Recursively apply heapify on the affected subtree

Example: Heap Sort using Array and Binary Heap Tree

Let's take a sample array:

Initial array: [4, 10, 3, 5, 1]

Step 1: Build a Max Heap from the array.

Convert the array into a binary tree and rearrange it into a Max Heap (largest element at the root).

heapify() function

Binary Tree Representation (Level Order):

```
    10
   / \
  5   3
 / \
4   1
```

Heap array after Max Heap construction:

[10, 5, 3, 4, 1]

Step 2: Sort the array by removing the root and placing it at the end

Swap root (largest) with last element:

[1, 5, 3, 4, 10]

Rebuild heap (ignore last element 10):

heap sort example

5
/\n
4 3
/
1

Heap array:

[5, 4, 3, 1, 10]

Repeat:

Swap root with 4th element:

[1, 4, 3, 5, 10]

heap sort example

4
/\n1 3

Heap array:

[4, 1, 3, 5, 10]

Swap root with 3rd element:

[3, 1, 4, 5, 10]

Heap array:

[3, 1, 4, 5, 10]

Swap root with 2nd element:

[1, 3, 4, 5, 10]

Final Sorted Array:

[1, 3, 4, 5, 10]

heap sort example

- `[[9, 4, 7, 1, -2, 6, 5]]`
-  MAX HEAP SORT (for Ascending Order)
-  Step 1: Build the Max Heap
- We use the heapify process from the last non-leaf node to the root.
- Tree from the array (index mapping):
- 9 (index 0)
- /
 \
 4 7 (1, 2)
- /
 \
 1 -2 6 5 (3, 4, 5, 6)

heap sort example

-  Heapify Steps
 - 1. $i = 2 \rightarrow \text{Node} = 7$
 - Children: 6 (index 5), 5 (index 6)
 - $7 > 6$ and $5 \rightarrow$ no change.
 - 2. $i = 1 \rightarrow \text{Node} = 4$
 - Children: 1 (3), -2 (4)
 - $4 > 1$ and $-2 \rightarrow$ no change.
 - 3. $i = 0 \rightarrow \text{Node} = 9$
 - Children: 4 (1), 7 (2)
 - $9 > \text{both} \rightarrow$ no change.

heap sort example

- 9
- /\
- 4 7
- /\ /\
- 1 -2 6 5
-  Step 2: Sort the Heap (Heap Sort Phase)
- Heap Size = 7
-  Iteration 1
- Swap 9 (root) and 5 (last) → [5, 4, 7, 1, -2, 6, 9]
- Heapify index 0

heap sort example

- Tree:
- 5
- / \
- 4 7
- / \ /
- 1 -2 6
- $7 > 5 \rightarrow \text{swap} \rightarrow [7, 4, 5, 1, -2, 6, 9]$
- Now 5 is at index 2 $\rightarrow$ children: 6 (index 5) $\rightarrow 6 > 5 \rightarrow \text{swap}$
- Result: $[7, 4, 6, 1, -2, 5, 9]$
-  Iteration 2

heap sort example

- Result: [6, 4, 5, 1, -2, 7, 9]
-  Iteration 3
- Swap 6 and -2 $\rightarrow$ [-2, 4, 5, 1, 6, 7, 9]
- Heapify 0 $\rightarrow$ max is 5 $\rightarrow$ swap
- Then heapify 2 $\rightarrow$ nothing to change
- Result: [5, 4, -2, 1, 6, 7, 9] $\rightarrow$ heapify again $\rightarrow$ max is 4 at index 1 $\rightarrow$ swap
- Then check index 1 $\rightarrow$ child 1 $>$ -2 $\rightarrow$ swap again
- Final: [5, 1, 4, -2, 6, 7, 9]
-  Continue...
- Eventually, we get:

heap sort example

Construct a heap for the list 1, 8, 6, 5, 3, 7, 4 by the bottom-up algorithm. Specify the property of building Max Heap. Maximum Key Deletion from a heap.

Build Max Heap (Bottom-Up)

Given list: 1, 8, 6, 5, 3, 7, 4

Step-by-step result:

Final Max Heap (Array):

[8, 5, 7, 1, 3, 6, 4]

Tree view:

```
      8
     / \
    5   7
   /\  /\
  1 3 6 4
```

heap sort example



2. Max Heap Property

- **Complete binary tree**
 - **Each parent $\geq$ its children**
 - **Root = maximum value**
-

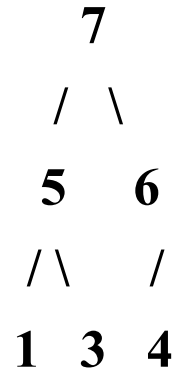


3. Delete Maximum (Root = 8)

- **Replace 8 with last value (4)**
- **Heap becomes: [4, 5, 7, 1, 3, 6]**
- **Heapify top-down $\rightarrow$ final heap: [7, 5, 6, 1, 3, 4]**

heap sort example

Tree after deletion:

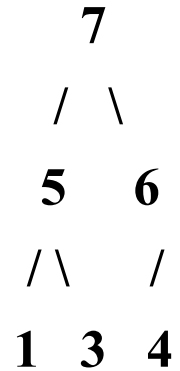


 4. Final Answer

- **Max Heap: [8, 5, 7, 1, 3, 6, 4]**
- **After deletion: [7, 5, 6, 1, 3, 4]**

heap sort example

Tree after deletion:



 4. Final Answer

- **Max Heap: [8, 5, 7, 1, 3, 6, 4]**
- **After deletion: [7, 5, 6, 1, 3, 4]**

3. Scheduling to Minimize Lateness

Problem Statement

- Given n jobs, each with a deadline and processing time, schedule them to minimize **maximum lateness**.
- **Greedy Algorithm (Earliest Deadline First)**
- Sort jobs by **deadline (ascending order)**.
- Schedule jobs in order of their deadlines.
- Compute lateness for each job

3. Scheduling to Minimize Lateness

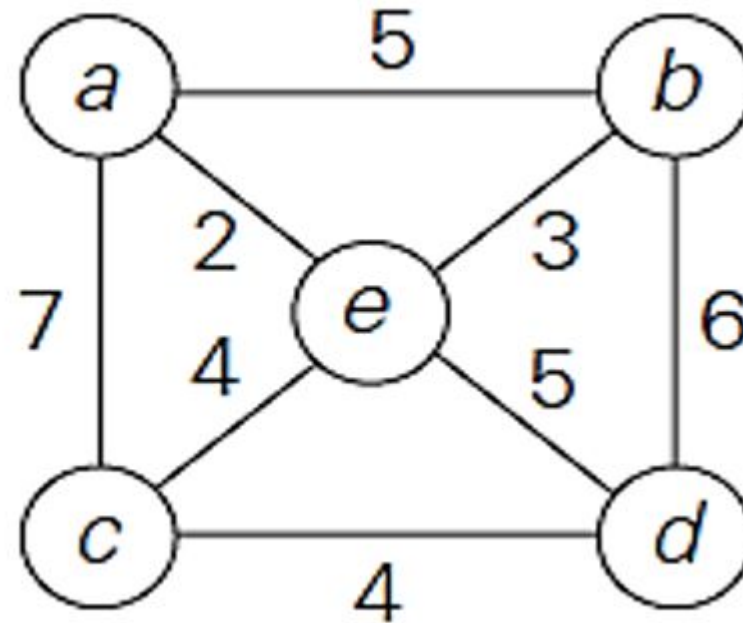
Job	Processing Time	Deadline
J1	2	5
J2	1	3
J3	4	6
J4	3	8

3. Scheduling to Minimize Lateness

- Schedule Order: $J2 \rightarrow J1 \rightarrow J3 \rightarrow J4$
- Compute maximum lateness: $L_{\max} = \max(\text{Completion Time} - \text{Deadline}, 0)$
- Time Complexity:
- Sorting takes $O(n \log n)$, and scheduling takes $O(n) \rightarrow O(n \log n)$ overall.

Kruskals

Apply Kruskal's algorithm to the graph given in Fig . Include in the priority queue all the vertices not already in the tree.



Kruskals

Vertices:

a, b, c, d, e

Edges and Weights:

Edge	Weight
a - e	2
b - e	3
c - e	4
c - d	4
a - b	5
d - e	5
b - d	6
a - c	7

Kruskals

Step 1: Sort All Edges by Weight

This acts like our priority queue (lowest weight first):

$$a - e = 2$$

$$b - e = 3$$

$$c - e = 4$$

$$c - d = 4$$

$$a - b = 5$$

$$d - e = 5$$

$$b - d = 6$$

$$a - c = 7$$

Kruskals

Step 2: Kruskal's Algorithm Steps

We use a Disjoint Set (Union-Find) to avoid cycles.

✓ Add edge a - e (2)

No cycle → Include

MST: {a, e}

Total weight: 2

✓ Add edge b - e (3)

b not connected → Include

MST: {a, b, e}

Total weight: 5

Kruskals

Add edge c - e (4)

c not connected → Include

MST: {a, b, c, e}

Total weight: 9

 **Add edge c - d (4)**

d not connected → Include

MST: {a, b, c, d, e} (All connected!)

Total weight: 13

Kruskals

-  Edge a - b (5) → Forms cycle → Skip
-  Edge d - e (5) → Forms cycle → Skip
-  Edge b - d (6) → Forms cycle → Skip
-  Edge a - c (7) → Forms cycle → Skip
-  Minimum Spanning Tree (MST)

Edge	Weight
------	--------

a - e	2
-------	---

b - e	3
-------	---

c - e	4
-------	---

c - d	4
-------	---

Total MST Weight = $2 + 3 + 4 + 4 = 13$

Shortest Path-Dijkstras Algorithm

Designing Dijkstra's Algorithm



Goal:

Minimize the total cost (weight) to reach each node from the source.



Data Structures Used:

`dist[]`: Stores the shortest distance from the source to each node.

`visited[]`: Marks nodes whose shortest distances are finalized.

Priority Queue (Min-Heap): To always process the node with the least current distance.



Algorithm Steps:

1. Set `dist[source] = 0`, and all others to ∞ .
2. Push (0, source) into a min-priority queue.

Shortest Path-Dijkstras Algorithm

3. While the queue is not empty:
4. Pop the vertex u with the minimum distance.
5. For each neighbor v of u , update $\text{dist}[v] = \min(\text{dist}[v], \text{dist}[u] + \text{weight}(u, v))$
6. Push updated distances into the priority queue

3. Analyzing the Algorithm:

Time Complexity $O((V + E) \log V)$ (with Min Heap)

Space Complexity $O(V)$ (for $\text{dist}[]$, $\text{visited}[]$, and PQ)

Type of Graph Weighted, directed or undirected

Edge Weights Non-negative only

4. Shortest Paths in a Graph

Problem Statement

- Find the shortest path from a **source node** to all other nodes in a weighted graph.
- **Dijkstra's Algorithm (Greedy Approach)**
- Set the source node's distance to **0**, and all others to ∞ .
- Select the **closest node** (minimum distance).
- Update its neighbors' distances if the new path is shorter.
- Repeat until all nodes are visited.

4. Shortest Paths in a Graph

4

(A) -----> (B)

| \

|

| \

| 2

1 |

\ 3

v

|

(C)----->(D)

|

1

v

(E)

4. Shortest Paths in a Graph

- If **A** is the source, the shortest paths:
- **$A \rightarrow B = 4$**
- **$A \rightarrow C = 3$**
- **$A \rightarrow D = 4$**
- **$A \rightarrow E = 1$**
- **Time Complexity:**
- Using a priority queue $\rightarrow O((V + E) \log V)$.

Prims Algorithm

- **Prim's algorithm** is a greedy algorithm used to find the Minimum Spanning Tree (MST) of a weighted, undirected graph.
- The Minimum Spanning Tree is a subset of the edges that connects all the vertices together, without any cycles, and with the minimum possible total edge weight.
- Steps of Prim's Algorithm:
- Start with any arbitrary node (vertex).
- Mark that node as part of the MST.
- Look at all the edges from the nodes included in MST to the nodes not yet included.
- Pick the edge with the smallest weight that connects a vertex inside the MST to a vertex outside.

Prims Algorithm

(A)

/ \

2 3

/ \

(B)-----1--(C)

Start with A.

A-B has weight 2, A-C has weight 3. Pick A-B (smaller).

Now $MST = \{A, B\}$.

From $\{A, B\}$, the smallest edge connecting to outside is B-C (weight 1).

Add B-C to MST.

$MST = \{A, B, C\}$ — Done.

Total weight = $2 + 1 = 3$.

Prims Algorithm

:Prim's Algorithm Complexity

Using a simple array:

Time complexity = $O(V^2)$ (where V = number of vertices)

Using a priority queue (min-heap) + adjacency list:

Time complexity = $O((V + E) \log V)$ (where E = number of edges)

Prims Algorithm

Vertices: 1, 2, 3, 4, 5

Edges (with weights):

(1)-(2): 1

(1)-(3): 2

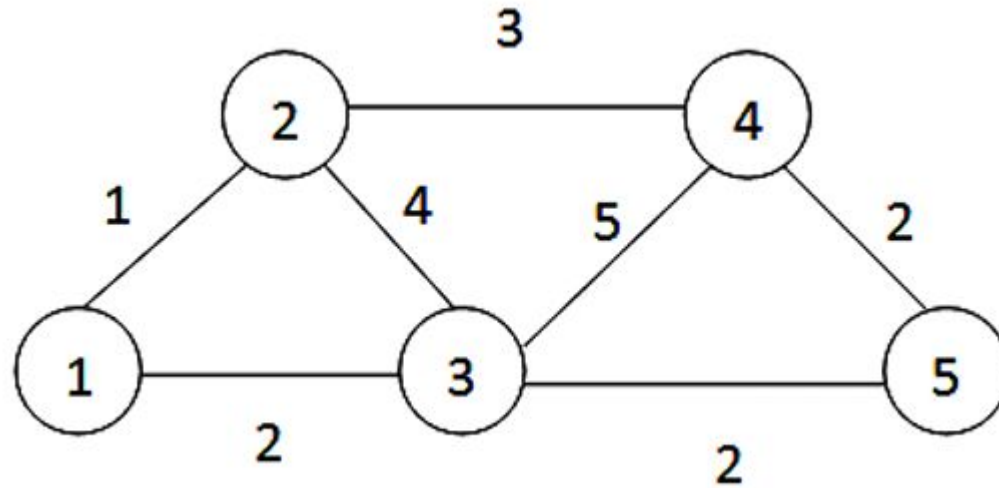
(2)-(4): 3

(2)-(3): 4

(3)-(4): 5

(3)-(5): 2

(4)-(5): 2



Prims Algorithm

✓ Now apply Prim's Algorithm starting from node 1:

Step 0: Start from Node 1

Add Node 1 to MST

Available edges from Node 1:

(1)-(2): weight 1

(1)-(3): weight 2

Pick the smallest → (1)-(2) with weight 1

✓ Edge selected: (1)-(2)

Step 1: Nodes in MST: {1, 2}

Available edges:

Prims Algorithm

(1)-(3): weight 2

(2)-(3): weight 4

(2)-(4): weight 3

Pick the smallest $\rightarrow$ (1)-(3) with weight 2

✓ Edge selected: (1)-(3)

Step 2: Nodes in MST: {1, 2, 3}

Available edges:

(2)-(4): weight 3

(3)-(4): weight 5

(3)-(5): weight 2

Prims Algorithm

Pick the smallest $\rightarrow$ (3)-(5) with weight 2

✓ Edge selected: (3)-(5)

Step 3: Nodes in MST: {1, 2, 3, 5}

Available edges:

(2)-(4): weight 3

(3)-(4): weight 5

(5)-(4): weight 2

Pick the smallest $\rightarrow$ (5)-(4) with weight 2

✓ Edge selected: (5)-(4)

🏆 Final MST Edges:

(1)-(2): weight 1

(1)-(3): weight 2

(3)-(5): weight 2

(5)-(4): weight 2

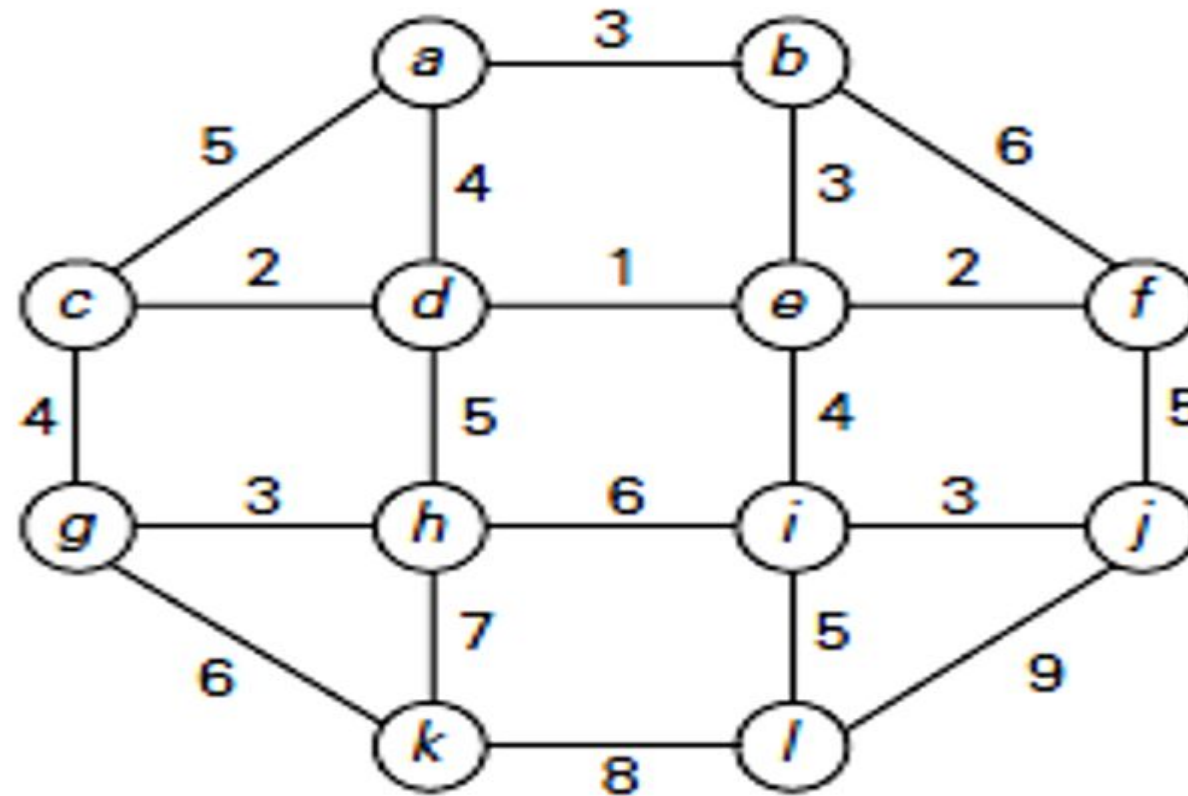
✏ **Total Weight of MST = $1 + 2 + 2 + 2 = 7$**

5. Minimum Spanning Tree (MST) Problem

- Find the subset of edges that connects all vertices with **minimum total edge weight** and **no cycles**.
- **Prim's Algorithm (Greedy Approach)**
 - Start with any node.
 - Pick the smallest edge that connects to an unvisited node.
 - Repeat until all nodes are connected.
- **Kruskal's Algorithm (Greedy + Sorting)**
 - Sort all edges by weight.
 - Pick the smallest edge that doesn't form a cycle.
 - Use **Union-Find** to detect cycles.

Kruskals Algorithm

Apply Kruskal's algorithm to the graph given in Fig. Include in the priority queue all the vertices already in the tree and show the traces.



Kruskals Algorithm

Step 1: List all edges with weights and sort them (ascending order)

Edge	Weight
d-e	1
e-f	2
c-d	2
b-e	3
i-j	3
a-b	3
g-h	3
a-d	4
c-g	4
e-i	4
d-h	5
c-a	5
h-i	6
b-f	6
g-k	6
k-h	7
i-l	5
k-l	8
j-l	9

Step 2: Initialize

- Each vertex is its own tree (disjoint sets)
- We'll pick the **smallest edge that doesn't form a cycle**

Step 3: Build MST (Minimum Spanning Tree)

- d-e (1) ✓
- c-d (2) ✓
- e-f (2) ✓
- b-e (3) ✓
- i-j (3) ✓
- a-b (3) ✓
- g-h (3) ✓

Kruskals Algorithm

Final MST Edges (in order added):

d-e (1)

c-d (2)

e-f (2)

b-e (3)

i-j (3)

a-b (3)

g-h (3)

e-i (4)

c-g (4)

k-h (7)

i-l (5)

5. Minimum Spanning Tree (MST) Problem

A --(1)-- B

| |

(2) (3)

| |

C --(4)-- D

- **MST (Kruskal's Algorithm):**
- Pick (A, B) [1], (A, C) [2], (B, D) [3]
- **Total Cost = 1 + 2 + 3 = 6**
- **Time Complexity:**
- **Kruskal's Algorithm** → **$O(E \log E)$** . due to edge sorting + union-find operations

5. Minimum Spanning Tree (MST) Problem

A --(1)-- B

| |

(2) (3)

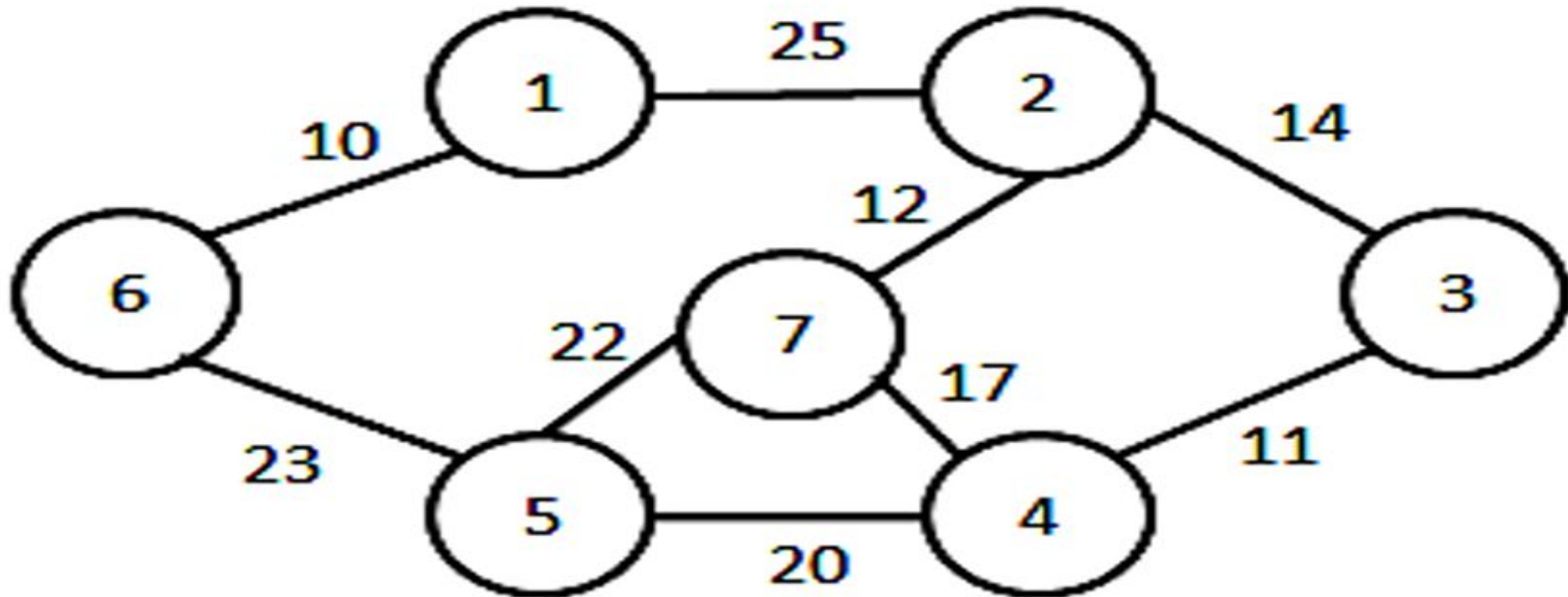
| |

C --(4)-- D

- **MST (Kruskal's Algorithm):**
- Pick (A, B) [1], (A, C) [2], (B, D) [3]
- **Total Cost = 1 + 2 + 3 = 6**
- **Time Complexity:**
- **Kruskal's Algorithm** → **$O(E \log E)$** . due to edge sorting + union-find operations

5. Minimum Spanning Tree (MST) Problem

Construct the minimum spanning tree for the graph given in Fig using Kruskal's algorithm.



5. Minimum Spanning Tree (MST) Problem

- **Step 1: List all the edges with weights**
- From the graph image, the edges with weights are:

Edge	Weight
(1,6)	10
(1,2)	25
(2,3)	14
(2,7)	12
(3,4)	11
(4,5)	20
(4,7)	17
(5,6)	23
(5,7)	22

5. Minimum Spanning Tree (MST) Problem

- **Step 2: Sort edges in increasing order of weights**

- Sorted edges:

1. $(1,6) \rightarrow 10$

2. $(3,4) \rightarrow 11$

3. $(2,7) \rightarrow 12$

4. $(2,3) \rightarrow 14$

5. $(4,7) \rightarrow 17$

6. $(4,5) \rightarrow 20$

7. $(5,7) \rightarrow 22$

8. $(5,6) \rightarrow 23$

5. Minimum Spanning Tree (MST) Problem

- **Step 3: Kruskal's Algorithm — Pick edges (no cycles)**
- We need $V - 1 = 7 - 1 = 6$ edges for MST.
- Let's begin:
-  Add (1,6) – 10 → no cycle
-  Add (3,4) – 11 → no cycle
-  Add (2,7) – 12 → no cycle
-  Add (2,3) – 14 → connects 2–3–4–7
-  Add (4,5) – 20 → no cycle
-  Add (5,6) – 23 → connects 1–6–5–4–2–3–7
-  Now all 7 nodes are connected with 6 edges — **MST complete!**

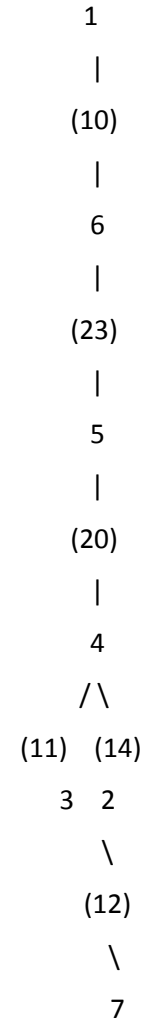
5. Minimum Spanning Tree (MST) Problem

- Final MST Edges

Edge	Weight
(1,6)	10
(3,4)	11
(2,7)	12
(2,3)	14
(4,5)	20
(5,6)	23

Total Weight of MST = $10 + 11 + 12 + 14 + 20 + 23 = 90$

5. Minimum Spanning Tree (MST) Problem



6. Huffman Coding and Data Compression

- 6. Huffman Coding and Data Compression
- Problem Statement
- Given a set of characters with **frequencies**, find an optimal way to encode them using **variable-length codes** to minimize the total encoding size.
- Huffman Coding Algorithm
- Create a **priority queue** with character frequencies.
- Pick the **two smallest** items and merge them into a new node.
- Repeat until we get a single tree.
- Assign **0** and **1** to left and right branches to generate codes.

6. Huffman Coding and Data Compression

- Example**

Characters: A(5), B(1), C(6), D(3), E(4)

- List characters with frequencies**

Character	Frequency
A	5
B	1
C	6
D	3
E	4

Step 2: Build the Huffman Tree

- We combine two smallest at each step:
- **B(1) + D(3) → Node with frequency 4**
- **E(4) + Node(4) → Node with frequency 8**
- **A(5) + C(6) → Node with frequency 11**
- **Node(8) + Node(11) → Root Node with 19**
- **Tree Structure (from bottom to top)**
- The final tree looks like this (as shown in the image):

```

    [19]
   /  \
  [8]  [11]

```

```

 /  \  /  \

```

E(4) [4] A(5) C(6)

6. Huffman Coding and Data Compression

Step 3: Assign Codes

Start from root, go:

- Left = 0
- Right = 1

Character	Path	Huffman Code
E	Left of 8	0
B	Left of 4 (which is right of 8)	1100
D	Right of 4 (right of 8)	1101
A	Left of 11 (right of root)	10
C	Right of 11 (right of root)	11

6. Huffman Coding and Data Compression

How to Assign Huffman codes stepwise:

- Every left branch = 0
- Every right branch = 1
- You start from the root and follow the path to the leaf node (character)
-  Let's walk through one character at a time:
-  Character E:
- E is directly on the left of the root → left = 0
- So E = 0

6. Huffman Coding and Data Compression

Character B:

To reach B:

From root → go left (0)

Then go right (1) to Node(4)

Then from Node(4), go left (0) to B

So path = 0 → 1 → 0

But from the image, it adds one more step (likely because Node(4) is a subtree under another internal node). So:

Full path: 1 (from root to right) → 1 (right again) → 0 → 0

So B = 1100

6. Huffman Coding and Data Compression

Character D:

Similar to B:

From root → left (0)

Then right (1) to Node(4)

Then right (1) to D

So D = 1101

6. Huffman Coding and Data Compression

★ Character A:

From root → right (1)

Then go left (0) to A

So A = 10

★ Character C:

From root → right (1)

Then go right (1) to C

So C = 11

Note: Follow the path from the root to the character. Every left turn is a 0, every right turn is a 1.
That's your Huffman code!

✓ Final Huffman Codes:

Character Code

A	10
B	1100
C	11
D	1101
E	0



Time Complexity:

If you use a priority queue, Huffman Coding takes

👉 $O(n \log n)$ time (as shown in the image).

-

Job sequencing

Consider a sequencing problem where there are four jobs having a profit of $(p_1, p_2, p_3, p_4) = (100, 10, 15, 27)$ and deadlines $(d_1, d_2, d_3, d_4) = (2, 1, 2, 3)$ respectively. Using Job Sequencing with deadlines find the solution for the above given instance. To solve follow these things

- Each job takes 1 unit of time.
- No two jobs can be scheduled at the same time.
- Each job must be completed before or on its deadline.
- The objective is to maximize total profit.

Step 1: Write down jobs with profit and deadline

Job	Profit	Deadline
J1	100	2
J2	10	1
J3	15	2
J4	27	3

Step 2: Sort jobs by profit (high to low)

Job sequencing

Step 3: Make time slots (based on max deadline = 3)

You have: Slot 1, Slot 2, Slot 3 → all empty

Step 4: Try to place each job in the latest free slot before its deadline

J1 → Deadline 2 → Slot 2 is free → Put J1 in Slot 2

J4 → Deadline 3 → Slot 3 is free → Put J4 in Slot 3

J3 → Deadline 2 → Slot 2 is taken → Try Slot 1 → Free → Put J3 in Slot 1

J2 → Deadline 1 → Slot 1 is taken → Cannot place

Step 5: Final Schedule

Slot Job

1 J3

2 J1

3 J4

 **Answer:**

Jobs selected: J3, J1, J4

Total Profit: $15 + 100 + 27 = ₹142$

